

Projeto Final - Matemática Discreta

Thiago Correa de Castro¹

¹Universidade Federal do Pará
Faculdade de Computação – Instituto de Ciências Exatas e Naturais
Av. Augusto Correa 01, 66075-090 – Belém – PA – Brasil

`thiago.correa@icen.ufpa.br`

Sumário

1	Introdução	4
2	Descrição do problema	5
2.1	O cenário	5
2.2	O objetivo	5
2.3	As regras	5
2.4	A Lógica Recursiva	5
3	Contextualização em um Cenário Prático	6
4	Explicação do código	6
4.1	Classe Config	6
4.1.1	Configurações dos Discos:	6
4.1.2	Dimensões Visuais (Geometria):	7
4.1.3	Velocidade de Animação:	7
4.1.4	Textos e Internacionalização:	7
4.1.5	Método get_text:	7
4.2	Classe Cores	8
4.2.1	Cores dos Discos:	8
4.2.2	Status:	8
4.3	Classe GameEngine	8
4.4	Classe HanoiSolver	9
4.5	Classe EventManager	10
4.6	Variável Global (Instância)	10
4.7	Classe HanoiApp	10
4.8	Bloco Principal (if __name__ == "__main__":)	12
4.9	Classe Disk	12
4.10	Classe Peg	12
4.11	Classe MoveCommand	13
4.12	Classe GameRenderer:	13
4.13	Classe RoundedPanel	14
4.14	Classe ModernButton	14
4.15	Classe StatBox	15

4.16	Classe Vector2D	15
4.17	Classe MathUtils	15
4.18	contador_linhas:	16
4.19	hanoi_benchmark(n, origem, destino, auxiliar):	16
4.20	rodar_bateria_testes(n_maximo):	16
4.21	Bloco Principal (if __name__ == "__main__"):	17
5	Comportamento Assintótico e Complexidade (Big-O)	17
5.1	Matemática do Código	18
5.1.1	Número de Chamadas de Função:	18
5.2	Prova Real com os dados da imagem:	19
5.3	O Limite Físico	19
5.3.1	Memória (Call Stack):	19
6	Conclusão	19

1. Introdução

A Torre de Hanói é um quebra-cabeça matemático inventado pelo matemático francês Édouard Lucas em 1883.

2. Descrição do problema

2.1. O cenário

Existem três hastes (ou pinos) verticais — vamos chamá-las de A (Origem), B (Auxiliar) e C (Destino). Na haste A, existe uma pilha de n discos de diâmetros diferentes. Os discos estão empilhados em ordem decrescente de tamanho, formando um formato cônico (o maior embaixo e o menor no topo).

2.2. O objetivo

Mover toda a pilha de discos da haste A para a haste C.

2.3. As regras

Para realizar a tarefa, deve-se obedecer a apenas três regras simples, mas rigorosas:

1. Apenas um disco pode ser movido por vez.
2. Cada movimento consiste em pegar o disco do topo de uma das pilhas e colocá-lo no topo de outra pilha.
3. Nenhum disco pode ser colocado sobre um disco menor que ele.

2.4. A Lógica Recursiva

O problema é perfeito para recursão porque, para mover uma pilha de tamanho 5, primeiro precisa-se saber como mover uma pilha de tamanho 4, e para isso a de tamanho 3, e assim por diante, até chegar ao caso base (mover 1 disco).

A estratégia se resume a:

1. Mover $n - 1$ discos da Origem para a Auxiliar.
2. Mover o maior disco (a base) da Origem para o Destino.
3. Mover os $n - 1$ discos da Auxiliar para o Destino.

3. Contextualização em um Cenário Prático

Tomemos como exemplo um centro de distribuição logística que utiliza um braço robótico para organizar contêineres ou caixas pesadas.

- O Problema: Você tem uma pilha de caixas onde as caixas mais pesadas e resistentes estão na base, e as mais leves e frágeis estão no topo. Você precisa mover essa pilha inteira de um caminhão de chegada (Origem) para uma área de expedição (Destino).
- A Restrição Física: Se o robô colocar uma caixa muito pesada em cima de uma caixa frágil (uma caixa maior sobre uma menor), a caixa de baixo será esmagada e o produto será destruído.
- A Restrição de Espaço: O armazém é apertado. Só se tem um espaço temporário livre (a haste Auxiliar) para colocar as caixas enquanto as reorganiza. O robô só consegue levantar uma caixa de cada vez.

O algoritmo da Torre de Hanói é a base para planejar a sequência de movimentos do braço robótico. Ele calcula o número mínimo de movimentos necessários para transferir a carga sem nunca violar a integridade física dos produtos (nunca colocar peso excessivo sobre uma base frágil), utilizando o espaço temporário de forma eficiente.

Isso transforma o problema de um "jogo de discos" para um problema real de Planejamento de Movimento (Motion Planning) e Restrição de Integridade em engenharia de sistemas.

4. Explicação do código

4.1. Classe Config

Esta classe define as configurações gerais de funcionamento, dimensões e textos do aplicativo. `class Config`: Inicia a definição da classe de configuração.

APP_TITLE: Define o título que aparecerá na barra superior da janela do aplicativo.

WIDTH: Define a largura da janela em pixels.

HEIGHT: Define a altura da janela em pixels.

FPS: Define a taxa de quadros por segundo (Frames Per Second), ou seja, a fluidez da animação.

4.1.1. Configurações dos Discos:

MIN_DISKS: O número mínimo de discos permitido no jogo.

MAX_DISKS: O número máximo de discos (para evitar que o jogo fique lento ou visualmente poluído).

DEFAULT_DISKS: O número de discos que aparecerá selecionado quando o programa abrir.

4.1.2. Dimensões Visuais (Geometria):

PEG_WIDTH: A largura das hastes (pinos) onde os discos são colocados.

PEG_HEIGHT: A altura dessas hastes.

BASE_HEIGHT: A altura da base retangular onde as hastes ficam apoiadas.

DISK_HEIGHT: A altura (espessura) de cada disco.

MIN_DISK_WIDTH: A largura do menor disco (o do topo).

MAX_DISK_WIDTH: A largura do maior disco (o da base). O programa provavelmente calcula o tamanho dos discos intermediários entre esses dois valores.

4.1.3. Velocidade de Animação:

Estas variáveis definem o tempo de espera (delay) entre movimentos.

ANIMATION_SPEED_SLOW: Velocidade lenta (2 segundos de pausa/movimento).

ANIMATION_SPEED_NORMAL: Velocidade normal.

ANIMATION_SPEED_FAST: Velocidade rápida.

ANIMATION_SPEED_INSTANT: Sem pausa (instantâneo).

4.1.4. Textos e Internacionalização:

TEXTS = {...}: Um dicionário que armazena os textos da interface. Isso prepara o código para ser traduzido facilmente (embora aqui só tenha pt_br).

Dentro de "pt_br", temos as chaves (ex: "start") e o texto que o usuário vê (ex: "Iniciar Solução Recursiva").

LANG = "pt_br": Define a linguagem atual como Português do Brasil.

4.1.5. Método get_text:

```
@staticmethod
def get_text(key: str) → str:
    return Config.TEXTS[Config.LANG].get(key, key)
```

- **@staticmethod:** Indica que este método pode ser chamado sem criar uma instância da classe (ex: `Config.get_text(...)`).
- **def get_text(key: str) → str:** Define a função que recebe uma chave de texto (string) e retorna uma string.
- **return Config.TEXTS[Config.LANG].get(key, key):**
 1. Acessa o dicionário TEXTS.
 2. Seleciona a linguagem definida em LANG (pt_br).

3. Usa `.get(key, key)` para buscar o texto. O segundo `key` é um valor padrão: se a tradução não for encontrada, ele retorna a própria chave (evitando que o programa quebre).

4.2. Classe Cores

Esta classe define a paleta de cores usada na interface gráfica. A paleta parece baseada no tema visual Nord (conhecido pelos tons frios e azulados).

class Cores: Inicia a classe de cores.

DARK_BG: Cor de fundo da janela (azul escuro quase preto).

DARK_PANEL: Cor para painéis ou áreas de menu.

DARK_ACCENT: Uma cor de destaque (azul ciano).

DARK_TEXT: Cor do texto (branco gelo), para contrastar com o fundo escuro.

DARK_PEG: Cor das hastes (cinza azulado).

DARK_BASE: Cor da base das hastes.

4.2.1. Cores dos Discos:

DISK_COLORS = [...]: Uma lista de códigos hexadecimais.

O programa provavelmente usará essas cores ciclicamente. Se houver 10 discos, cada um terá uma cor diferente para facilitar a visualização. As cores variam entre vermelho, laranja, amarelo, verde e tons de azul/roxo.

4.2.2. Status:

SUCCESS: Verde (para mensagens de sucesso).

ERROR: Vermelho (para mensagens de erro, como movimento inválido).

WARNING: Amarelo (para avisos).

4.3. Classe GameEngine

Esta classe é responsável por gerenciar o "tabuleiro". Ela controla os pinos (pegs), os discos, o histórico de jogadas e garante que as regras da Torre de Hanói sejam respeitadas.

__init__(self): É o construtor da classe. Ele inicializa as variáveis de controle (como o número de discos, contagem de movimentos, histórico e flags de controle) e, em seguida, chama as funções internas para montar e reiniciar o jogo.

_setup_pegs(self): Função interna que cria os três pinos clássicos do jogo: Origem (A), Auxiliar (B) e Destino (C), já definindo de forma proporcional onde eles ficarão na tela.

reset_game(self, new_disk_count): Limpa e reinicia o estado atual do jogo. Remove todos os discos dos pinos, zera os contadores e recria os discos empilhados em

ordem decrescente (do maior na base ao menor no topo) no pino de Origem. Por fim, avisa o sistema que o jogo foi resetado.

move_disk(self, source_idx, target_idx): Contém a lógica central e de validação do jogo. Ela tenta mover um disco de um pino de origem para um pino de destino.

- Verifica se o pino de origem realmente tem discos.
- Aplica a regra de ouro do jogo: um disco maior não pode ser colocado em cima de um disco menor.
- Se o movimento for válido, ele retira o disco da origem, coloca no destino, adiciona o movimento no histórico, conta a jogada e checa se todos os discos chegaram ao último pino (condição de vitória).

update_peg_positions(self, width, height): Usado para recalcular a posição dos pinos caso a tela ou janela do jogo mude de tamanho (responsividade), garantindo que os pinos continuem com o espaçamento correto.

4.4. Classe HanoiSolver

Esta classe funciona como um "bot" para o jogo. Ela implementa o algoritmo de resolução automática de forma assíncrona (usando threads) para não congelar a interface visual do jogo enquanto pensa ou realiza os movimentos.

__init__(self, engine): Recebe a instância do GameEngine para poder manipular as peças. Também define a velocidade padrão em que a animação da solução vai rodar e prepara a variável da thread de execução.

start_solving(self): Inicia o processo automático. Se o jogo já estava começado por um jogador, ele reseta o tabuleiro para iniciar do zero limpo. Depois, ele cria uma Thread para rodar a solução em segundo plano, não atrapalhando os outros processos visuais.

stop(self): Permite que o usuário interrompa a resolução automática a qualquer momento. Ela altera a flag stop_requested para True, o que avisa o algoritmo para parar o que estiver fazendo.

set_speed(self, speed_val): Ajusta o atraso de tempo (delay) entre os movimentos automáticos, permitindo acelerar ou desacelerar a animação visual.

_run_thread(self): É o ponto de entrada da thread em segundo plano. Marca o horário de início, chama a função recursiva que resolve de fato o jogo e, quando termina, calcula o tempo total gasto emitindo uma mensagem de finalização.

_recursive_move(self, n, source, target, aux): É o cérebro do solucionador. Ele implementa a famosa solução matemática recursiva da Torre de Hanói:

1. Move $n - 1$ discos da Origem para o pino Auxiliar.
 2. Move o maior disco (base) para o Destino.
 3. Move os $n - 1$ discos do Auxiliar para o Destino.
- Detalhe importante: Antes de cada passo, ela checa se stop_requested é verdadeiro. Isso evita que o algoritmo fique em loop caso o usuário queira cancelar no meio da solução, além de utilizar o time.sleep para dar o efeito animado.

4.5. Classe EventManager

É a classe principal responsável por gerenciar e despachar os eventos do sistema.

__init__(self): É o construtor da classe. Ele inicializa o atributo `self._listeners`, que é um dicionário (`Dict[str, List[Callable]]`). As chaves desse dicionário são os nomes dos eventos (strings, como "move_start" ou "reset"), e os valores são listas de funções (callbacks) que devem ser executadas quando esse evento ocorrer.

subscribe(self, event_type: str, callback: Callable): Este método permite que outras partes do código se "inscrevam" para escutar um evento específico.

- Ele recebe o nome do evento (`event_type`) e a função que deve ser chamada (`callback`).
- Se o evento ainda não existir no dicionário, ele cria uma nova lista vazia para ele.
- Em seguida, adiciona a função `callback` à lista de ouvintes daquele evento.

emit(self, event_type: str, data: Any = None): Este método é usado para "disparar" (ou emitir) um evento, notificando todos os inscritos.

- Ele recebe o nome do evento (`event_type`) e, opcionalmente, algum dado associado a esse evento (`data`).
- Ele verifica se o evento existe no dicionário `self._listeners`.
- Se existir, ele percorre a lista de funções (callbacks) registradas para aquele evento e as executa, passando a variável `data` como argumento.
- O bloco `try...except` garante que, se uma das funções inscritas gerar um erro, o programa apenas imprimirá a mensagem de erro no console (`print(f"Erro no evento event_type: e")`) e continuará executando os próximos callbacks, sem travar a aplicação inteira.

4.6. Variável Global (Instância)

events = EventManager(): No final do arquivo, é criada uma instância global da classe `EventManager` chamada `events`. Isso serve para que todo o projeto possa importar e compartilhar exatamente a mesma instância de gerenciamento (funcionando de forma similar a um Singleton). Dessa forma, se a interface gráfica (`main.py`) se inscrever em um evento e o motor do jogo (`engine.py`) emitir esse mesmo evento, ambos estarão utilizando o mesmo canal de comunicação.

4.7. Classe HanoiApp

Esta é a única classe definida no arquivo e ela herda de `tk.Tk` (a classe principal do `tkinter` para criação de janelas). Ela representa a janela principal da aplicação e gerencia toda a interface com o usuário. Métodos da Classe:

__init__(self): É o construtor da classe. Ele inicializa a janela do aplicativo, define seu título, dimensões padrão, tamanho mínimo e cor de fundo. Também instancia o motor do jogo (`GameEngine`) e o solucionador automático (`HanoiSolver`), chama os métodos para construir a interface visual (`_setup_ui`) e vincular os eventos globais (`_bind_events`), finalizando com um registro de log informando que o sistema iniciou.

_setup_ui(self): Responsável por construir e posicionar todos os elementos gráficos da janela. Ele cria o cabeçalho superior, a barra lateral direita (contendo configurações, sliders de velocidade e número de discos, botões de ação e quadro de estatísticas),

uma caixa para os logs de execução e a área central onde a torre é renderizada usando `GameRenderer`.

`_bind_events(self)`: Conecta o sistema de eventos (`EventManager`) aos métodos da interface. Isso permite que, quando algo acontece na lógica (por exemplo, início de um movimento, erros, jogo finalizado ou logs gerados), a interface gráfica (UI) saiba como reagir automaticamente através dos métodos definidos.

`on_disk_change(self, val)`: É a função de callback acionada quando o usuário move o controle deslizante (slider) de quantidade de discos. Se o solucionador não estiver resolvendo o jogo ativamente, esta função reseta o jogo com o novo número de discos, atualiza a exibição de quantidade de movimentos mínimos necessários e redesenha o tabuleiro.

`on_speed_change(self, val)`: Função de callback acionada pelo controle deslizante de velocidade. Ela converte o valor visual selecionado em um tempo de atraso (delay) matemático e o repassa para o solucionador ajustar a velocidade da animação em tempo real.

`action_solve(self)`: Executada ao clicar no botão "Iniciar Solução Recursiva" ou "Parar". Ela avalia se o jogo já está em execução; caso esteja, solicita a parada e limpa o botão. Se não estiver, dá início ao algoritmo de solução do `HanoiSolver`.

`action_reset(self)`: Executada ao clicar no botão de "Reiniciar Tabuleiro". Para a execução do algoritmo caso esteja rodando, aplica um breve intervalo de 100 milissegundos, limpa as posições dos pinos recriando a torre inicial e manda o renderizador atualizar a tela.

`on_move_event(self, move_cmd: MoveCommand)`: Recebe o evento de que um movimento vai começar. Atualiza a caixa de estatística mostrando o novo total de movimentos computados e aciona a animação de deslocamento do disco através da classe `GameRenderer`.

`on_reset_event(self, _)`: É acionada pelo evento global de "reset". Sua função é zerar visualmente o contador de movimentos, alterar o texto de status de volta para "Pronto" e atualizar o renderizador gráfico.

`update_status(self, text)`: Função utilitária que recebe uma string e altera o texto do componente de etiqueta (label) que exibe o status atual do jogo na interface.

`log(self, message)`: Formata uma mensagem adicionando a hora atual ([HH:MM:SS]), a exibe no console através de um print e também a insere na lista visual de logs (`log_box`) dentro do aplicativo, rolando automaticamente para o final da lista para mostrar o evento mais recente.

`show_error(self, msg)`: Aciona uma janela de alerta do sistema (messagebox) exibindo o erro passado como argumento e o título "Erro".

`on_closing(self)`: Método disparado quando o usuário tenta fechar a janela da aplicação (no "X"). Ele garante uma finalização segura, parando o algoritmo de solução, informando o renderizador para parar as animações e, por fim, destruindo a janela do `tkinter`.

4.8. Bloco Principal (if `__name__ == "__main__":`)

Este é o bloco condicional que determina o que fazer caso o script `main.py` seja o arquivo executado diretamente.

- Primeiro, ele tenta ativar o suporte a alto DPI (`DpiAwareness`) específico para computadores com sistema Windows usando o `ctypes`, visando manter a interface mais nítida em telas modernas.
- Em seguida, cria a instância global da janela chamando `HanoiApp()`.
- Conecta o evento padrão de fechar janela (`WM_DELETE_WINDOW`) ao método próprio `on_closing`.
- Por fim, invoca o `app.mainloop()`, que é o laço infinito do tkinter responsável por manter a janela aberta e ouvir ativamente os eventos do mouse e teclado do usuário.

4.9. Classe Disk

Esta classe representa um disco individual no jogo.

`__init__(self, size_index: int, total_disks: int)`: É o construtor da classe. Ele inicializa o disco com um identificador (`id`), calcula sua largura dinamicamente com base no número total de discos (`MathUtils.get_disk_width`), e atribui uma cor da paleta definida nas configurações. Também inicializa variáveis para controlar a posição atual do disco (`pos`), a posição para onde ele deve ir (`target_pos`) e um sinalizador para indicar se ele está atualmente em uma animação de movimento (`is_moving`).

`__repr__(self)`: Retorna uma representação em formato de texto do objeto (ex: `Disk(0)`), o que facilita a leitura durante a depuração do código.

4.10. Classe Peg

Esta classe representa um dos pinos (ou hastes) do tabuleiro onde os discos são empilhados.

`__init__(self, peg_id: int, name: str, center_x: int, base_y: int)`: Inicializa o pino com um identificador (`id`), um nome de exibição (como "Origem (A)" ou "Destino (C)") e suas coordenadas centrais no eixo X e na base do eixo Y. Ele também cria uma lista vazia `self.disks` que funcionará como a pilha de discos daquele pino.

`push(self, disk: Disk) → bool`: Tenta adicionar um disco ao topo do pino. Ele aplica as regras do jogo verificando se a pilha está vazia ou se o disco sendo adicionado é menor que o disco atualmente no topo (`self.disks[-1].id > disk.id`). Se for um movimento válido, o disco é adicionado, sua nova posição de repouso é calculada, e a função retorna `True`. Caso contrário, retorna `False`.

`pop(self) → Optional[Disk]`: Remove e retorna o disco que está no topo do pino. Se o pino estiver vazio, ele apenas retorna `None`.

`peek(self) → Optional[Disk]`: Apenas "espia" o disco no topo do pino e o retorna sem removê-lo da lista.

`is_empty(self) → bool`: Uma função utilitária que retorna `True` se não houver nenhum disco empilhado no pino.

__update_disk_target(self, disk: Disk): Uma função interna usada para calcular matematicamente a posição Y alvo do disco para que ele fique empilhado corretamente em cima dos outros discos já existentes no pino.

refresh_layout(self): Percorre todos os discos atuais do pino para recalcular e atualizar suas posições alvo. Se um disco não estiver no meio de uma animação de movimento (`is_moving == False`), sua posição atual é instantaneamente atualizada para a posição alvo correta.

4.11. Classe MoveCommand

Esta classe funciona como um registro histórico de uma ação executada no jogo.

__init__(self, disk: Disk, source_peg: Peg, target_peg: Peg): Cria um registro que armazena exatamente qual disk foi movido, de qual pino de origem (`source_peg`) para qual pino de destino (`target_peg`). Além disso, captura o instante exato em que o movimento foi criado usando `time.time()`.

__str__(self): Retorna uma string descritiva e amigável daquele movimento, como por exemplo: "Disco 1 de Origem (A) para Auxiliar (B)".

4.12. Classe GameRenderer:

__init__(self, parent, engine: GameEngine): É o método construtor da classe que inicializa o componente visual (Canvas) definindo cores de fundo e removendo bordas. Associa a lógica do jogo (`engine`), cria uma lista para rastrear os discos que estão em movimento (`animating_disks`), vincula o evento de redimensionamento da janela (`<Configure>`) e inicia o loop contínuo de renderização chamando o método `_animate()`.

on_resize(self, event): Evento acionado quando a janela do programa muda de tamanho e chama a função da lógica (`engine.update_peg_positions`) para recalcular a posição dos pinos com base na nova largura e altura disponíveis, e em seguida solicita o redesenho imediato da tela usando `redraw()`.

_animate(self): É o loop principal responsável por manter o jogo animado quadro a quadro. Desde que a flag `self.running` seja verdadeira, a função atualiza as posições dos elementos (`update_animations()`), pinta o canvas novamente (`redraw()`) e usa o método assíncrono `self.after` do Tkinter para se chamar novamente, calculando o atraso com base na constante `Config.FPS` para manter a fluidez de quadros.

start_disk_animation(self, disk: Disk, from_pos: Vector2D, to_pos: Vector2D, duration: float): Função que prepara um disco para iniciar seu movimento de um pino para outro que cria uma trajetória baseada em 4 pontos (vetores). O disco sobe do pino de origem (`p1` para `p2`), se move horizontalmente até o topo do pino de destino (`p2` para `p3`) e, finalmente, desce até sua posição no pino de destino (`p3` para `p4`). O disco e suas variáveis de controle de animação são adicionados à lista `animating_disks`.

update_animations(self): Executa a matemática por trás da mudança de posição dos discos a cada quadro. Percorre a lista de todos os discos sendo animados e avança a variável de progressão do tempo (`t`). Se o progresso atingir 1.0, o disco passa para a próxima etapa da sua rota de quatro pontos. O método utiliza funções do arquivo de utilidades (como `MathUtils.ease_in_out_cubic` e `MathUtils.lerp`) para aplicar efeito de

"easing"(aceleração e desaceleração suaves) e interpolação linear nas coordenadas X e Y. Discos que terminam o percurso são removidos da lista e marcados como parados.

redraw(self): Limpa e refaz o desenho visual estático de todos os elementos na interface. Primeiro apaga tudo do canvas (`self.delete("all")`). Depois, constrói a base inferior de suporte dos pinos. Em seguida, desenha as torres verticais (pinos) usando suas coordenadas (`base_y`, `center_x`) e coloca o nome abaixo delas. Por último, consolida todos os discos de todos os pinos e aciona a função auxiliar `_draw_disk` para exibi-los em suas posições atuais.

_draw_disk(self, disk: Disk): Método auxiliar privado para criar os elementos visuais de apenas um disco. Com base na posição (X e Y), tamanho e cor contidos na classe `Disk`, esta função desenha três partes:

- O retângulo principal preenchido com a cor do disco.
- Uma linha clara na parte superior interna que atua como um destaque de luz para criar uma noção de volume.
- Um número no centro correspondente à identificação/tamanho do disco.

4.13. Classe `RoundedPanel`

Esta classe herda de `tk.Canvas` e tem como objetivo criar um painel (um fundo) com cantos arredondados, dando um aspecto mais moderno à interface.

__init__(self, parent, width, height, bg_color, radius=15, **kwargs): É o construtor da classe. Ele inicializa o canvas sem bordas de destaque, define propriedades como cor de fundo (`bg_color`), raio da borda arredondada (`radius`, com valor padrão de 15), largura e altura, e por fim chama o método `draw()` para renderizar a forma.

draw(self): Limpa o canvas e desenha o painel arredondado. Para obter o formato de cantos arredondados, ele desenha quatro arcos (`create_arc`) que funcionam como os cantos da figura e usa retângulos (`create_rectangle`) para preencher os espaços horizontais e verticais entre esses arcos.

4.14. Classe `ModernButton`

Herda de `tk.Canvas` e implementa um botão interativo customizado que reage quando o mouse passa por cima (`hover`) ou clica nele.

__init__(self, parent, text, command, width=120, height=40, bg_color="#4C566A", hover_color="#5E81AC"): Construtor que inicializa as propriedades do botão, como o texto exibido, a função que ele deve executar (`command`), suas dimensões e suas cores de estado normal e de interação. Ele também associa (realiza o `bind`) os eventos do mouse aos seus métodos correspondentes e faz o desenho inicial.

draw(self, color, offset=0): Apaga o desenho atual e recria o botão com a cor informada. Se houver um valor de `offset` maior que zero, o botão é desenhado levemente deslocado para simular o efeito visual de estar sendo pressionado. O texto é sempre centralizado.

on_enter(self, e): Função acionada quando o cursor do mouse entra na área do botão. Redesenha o botão usando a cor de destaque (`bg_hover`).

on_leave(self, e): Função acionada quando o cursor do mouse sai do botão. Redesenha o botão restaurando a cor original (bg_normal).

on_click(self, e): Função acionada no momento em que o botão do mouse é clicado. Invoca o método draw passando um offset=2 para aplicar o efeito visual de "afundar" o botão.

on_release(self, e): Função acionada quando o clique é solto. Restaura o botão para a posição original de hover e, se o botão possuir um comando (self.command), ele o executa.

4.15. Classe StatBox

Herda de tk.Frame e serve como um pequeno contêiner para exibir informações na tela (como uma caixa de estatísticas).

__init__(self, parent, title, value="0"): Configura o fundo do frame e instancia dois textos na tela (tk.Label). O primeiro exibe o título (lbl_title) com uma fonte menor; o segundo exibe o valor numérico em si (lbl_value) com uma fonte maior e em negrito.

set_value(self, val): Atualiza dinamicamente o texto da caixa de valor numérico (lbl_value) convertendo o dado que foi passado (val) em formato string.

4.16. Classe Vector2D

Esta classe representa um vetor ou ponto no espaço bidimensional (2D), contendo as coordenadas x e y. Ela é utilizada para controlar a posição e a trajetória dos discos durante as animações.

__init__(self, x: float, y: float): É o construtor da classe. Ele inicializa o vetor com os valores das coordenadas horizontais (x) e verticais (y).

__add__(self, other): É um método mágico que sobrecarrega o operador de adição (+). Ele permite somar dois objetos Vector2D, retornando um novo vetor cujas coordenadas são a soma das coordenadas originais.

__sub__(self, other): Sobrecarrega o operador de subtração (-). Permite subtrair um vetor de outro, retornando um novo vetor com as diferenças das coordenadas.

__mul__(self, scalar): Sobrecarrega o operador de multiplicação (*). Permite multiplicar o vetor por um número escalar (um único valor numérico), alterando a magnitude do vetor.

distance_to(self, other) → float: Calcula e retorna a distância Euclidiana (em linha reta) entre o vetor atual e um outro vetor passado como parâmetro usando o Teorema de Pitágoras.

to_tuple(self) → Tuple[float, float]: Converte o objeto do vetor em uma tupla simples contendo (x, y).

__repr__(self): Retorna uma representação em formato de string do vetor (ex: Vec2(10.0, 20.0)). Isso é útil principalmente para debug (encontrar erros) no código.

4.17. Classe MathUtils

Esta classe funciona como uma coleção de funções matemáticas estáticas (que não precisam de uma instância da classe para serem chamadas) usadas para cálculos de animação

e dimensionamento.

lerp(start: float, end: float, t: float) → float: Implementa a Interpolação Linear (Linear Interpolation). Ela calcula e retorna um valor intermediário entre um ponto de partida (start) e um ponto de chegada (end) com base em um percentual t (normalmente entre 0.0 e 1.0). É amplamente usada para mover os discos quadro a quadro na tela.

ease_in_out_cubic(t: float) → float: Implementa uma função de suavização de movimento (easing). Em vez de o disco se mover em uma velocidade constante do início ao fim, essa função cúbica faz com que o movimento comece de forma mais lenta, acelere no meio, e desacelere suavemente ao chegar no destino, tornando a animação muito mais natural.

get_disk_width(disk_index: int, total_disks: int) → int: É responsável por calcular a largura visual (em pixels) de cada disco. Ela pega a largura mínima e máxima permitidas (definidas no arquivo Config), e calcula a largura ideal do disco atual com base no seu índice (se é o disco menor, médio, maior) e na quantidade total de discos sendo jogados naquela partida, garantindo que a pirâmide sempre fique proporcional.

4.18. contador_linhas:

Esta variável é inicializada com o valor 0 no escopo global do arquivo.

Ela é utilizada para rastrear e contabilizar a quantidade de operações ou linhas executadas durante o processo de resolução da Torre de Hanói.

4.19. hanoi_benchmark(n, origem, destino, auxiliar):

É uma função recursiva que simula a lógica de movimentação dos discos na Torre de Hanói.

Ao ser chamada, a primeira ação que realiza é acessar a variável global `contador_linhas` e incrementá-la em 1.

A função verifica se o número de discos `n` é maior que 0. Se for, ela realiza uma chamada recursiva para mover `n - 1` discos da origem para o pino auxiliar, incrementa o `contador_linhas` em 1 novamente para representar o movimento do disco atual, e faz outra chamada recursiva para mover os `n - 1` discos do pino auxiliar para o destino.

4.20. rodar_bateria_testes(n_maximo):

Esta função gerencia a execução dos testes, recebendo o parâmetro `n_maximo`, que define o limite de discos a serem testados.

Ela gera uma lista chamada `valores_n` contendo a sequência de números de 1 até o valor de `n_maximo` e cria uma lista vazia chamada `historico_linhas`.

A função itera sobre cada número de discos (`n`) presente na lista, zerando a variável global `contador_linhas` a cada repetição antes de invocar a função `hanoi_benchmark`.

Após a execução do algoritmo, ela armazena o total de operações no `historico_linhas` e imprime uma mensagem no terminal informando a quantidade de discos e as operações executadas correspondentes.

Por fim, a função utiliza a biblioteca `matplotlib.pyplot` para plotar um gráfico visual do desempenho. O gráfico exibe os "Valores de n " no eixo X e o "Histórico de Linhas" no eixo Y, contendo um título explicativo, rótulos para os eixos, grade transparente ($\alpha=0.7$) e uma linha na cor `#BF616A` marcando os pontos.

4.21. Bloco Principal (if `__name__ == "__main__":`):

Este bloco garante que o código de teste seja executado apenas se o arquivo for rodado diretamente como script.

Ele chama a função `rodar_bateria_testes(15)`, indicando que, por padrão, o script realiza a análise de desempenho calculando a complexidade do algoritmo da Torre de Hanói para configurações de 1 até 15 discos.

5. Comportamento Assintótico e Complexidade (Big-O)

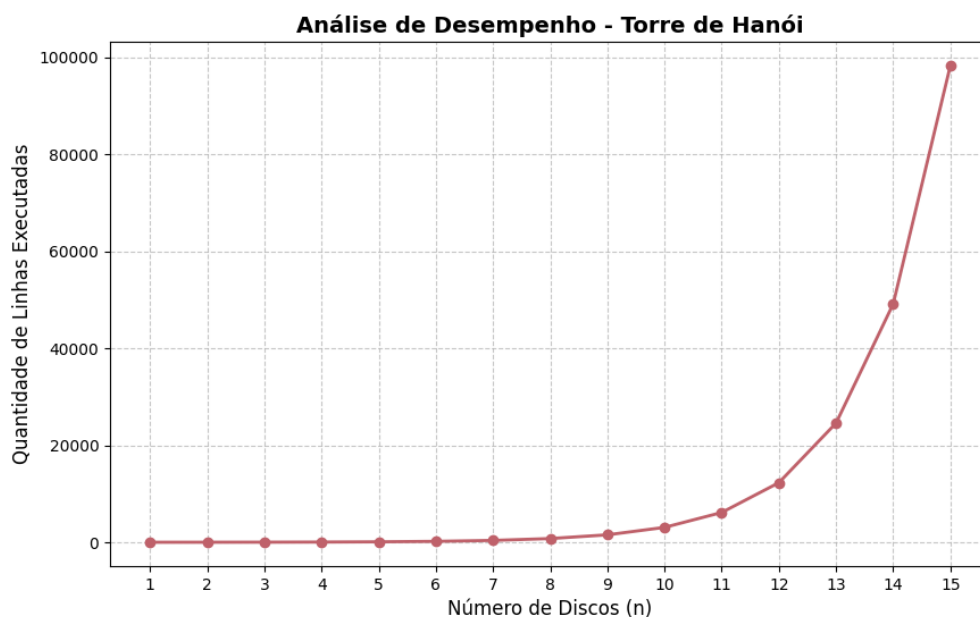


Figura 1. Gráfico de complexidade

O gráfico gerado (`gráfico.png`) exibe a clássica curva de um algoritmo de Complexidade de Tempo Exponencial, denotada por $O(2^n)$.

Nesta classe de complexidade, o esforço computacional (neste caso, medido pelo número de linhas/operações executadas) dobra a cada nova unidade adicionada à entrada (cada novo disco).

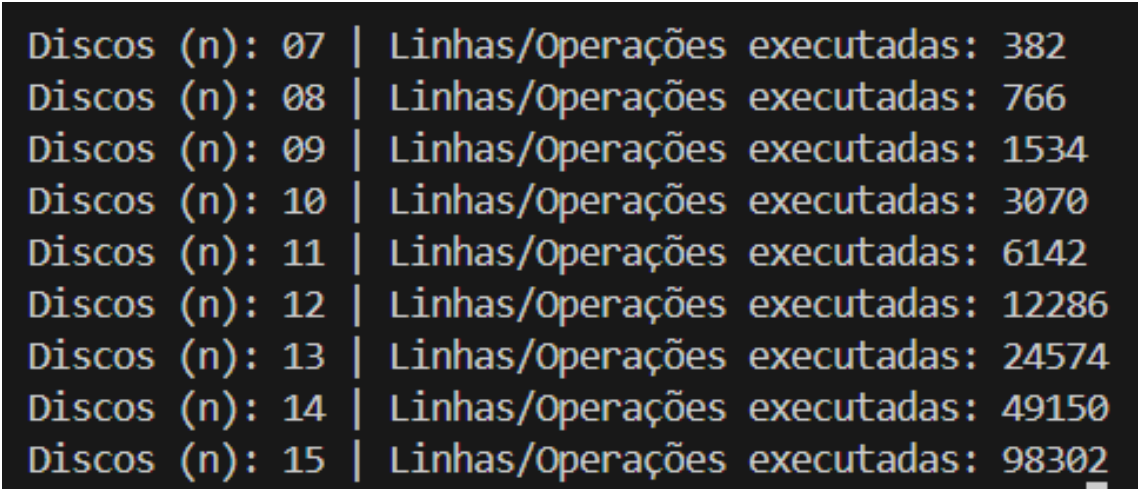
Observando o gráfico, fica evidente a natureza traiçoeira do crescimento exponencial:

Fase Plana (Início): Para valores pequenos ($n < 7$), a curva parece quase colada ao eixo X. O computador resolve o problema de forma imperceptível.

Ponto de Inflexão Visual: A partir de $n = 10$, a curva começa a se inclinar drasticamente para cima.

Crescimento Explosivo: Entre $n = 12$ e $n = 15$, a linha torna-se quase vertical. Adicionar apenas 3 discos faz o número de operações saltar de ≈ 12.000 para quase 100.000.

5.1. Matemática do Código



Discos (n): 07		Linhas/Operações executadas: 382
Discos (n): 08		Linhas/Operações executadas: 766
Discos (n): 09		Linhas/Operações executadas: 1534
Discos (n): 10		Linhas/Operações executadas: 3070
Discos (n): 11		Linhas/Operações executadas: 6142
Discos (n): 12		Linhas/Operações executadas: 12286
Discos (n): 13		Linhas/Operações executadas: 24574
Discos (n): 14		Linhas/Operações executadas: 49150
Discos (n): 15		Linhas/Operações executadas: 98302

Figura 2. Retorno de benchmark.py no terminal

A imagem execuções.png revela números exatos muito interessantes, como $n=10$ resultando em 3070 operações e $n=15$ resultando em 98302 operações.

Mas de onde vêm exatamente esses números? Vamos analisar o código do benchmark.py:

```
def hanoi\_benchmark(n, origem, destino, auxiliar):  
    global contador\_linhas  
    contador\_linhas += 1 \# Conta 1: Chamada da função  
  
    if n > 0:  
        hanoi\_benchmark(n - 1, origem, auxiliar, destino)  
        contador\_linhas += 1 \# Conta 2: O "movimento" do disco  
        hanoi\_benchmark(n - 1, auxiliar, destino, origem)
```

O script incrementa o contador em dois momentos: a cada chamada de função e a cada movimento de disco.

Número de Movimentos: A matemática nos diz que o número mínimo de movimentos para resolver a Torre de Hanói é $M(n) = 2^n - 1$.

5.1.1. Número de Chamadas de Função:

Uma árvore de recursão binária completa para a Torre de Hanói tem um total de chamadas equivalente a $C(n) = 2^{n+1} - 1$.

Somando ambos os incrementos, temos a função exata de linhas executadas pelo script:

$$\begin{aligned}L(n) &= M(n) + C(n) \\L(n) &= (2^n - 1) + (2^{n+1} - 1) \\L(n) &= 2^n + 2 \cdot 2^n - 2 \\L(n) &= 3 \cdot 2^n - 2\end{aligned}$$

5.2. Prova Real com os dados da imagem:

Para $n = 10$: $3 \cdot 2^{10} - 2 = 3 \cdot 1024 - 2 = 3072 - 2 = \mathbf{3070}$

Para $n = 15$: $3 \cdot 2^{15} - 2 = 3 \cdot 32768 - 2 = 98304 - 2 = \mathbf{98302}$

Os dados empíricos do benchmark batem perfeitamente com o modelo matemático preditivo.

5.3. O Limite Físico

Apesar de o computador resolver $n=15$ numa fração de segundo, o algoritmo de Hanói é um excelente exemplo de como rapidamente atingimos os limites físicos do hardware:

Para $n = 30$: Seriam necessários mais de 3 bilhões de operações. Em Python puro, isso já causaria um congelamento perceptível de vários segundos ou minutos.

A Lenda de Brahma ($n = 64$): A lenda original propõe uma torre de 64 discos de ouro. Usando a fórmula do benchmark, isso resultaria em $\approx 5.5 \times 10^{19}$ operações. Mesmo se um supercomputador pudesse executar 1 bilhão de operações por segundo, ele levaria cerca de 1.7 bilhões de anos para concluir o cálculo.

5.3.1. Memória (Call Stack):

No Python, a profundidade máxima de recursão padrão é de 1000. Embora a profundidade da árvore (uso de memória) da Torre de Hanói seja apenas $O(n)$ (ou seja, a memória cresce linearmente, não exponencialmente), em linguagens sem otimização de recursão em cauda, tentar resolver para valores como $n=2000$ causaria um erro de `RecursionError` muito antes de o tempo estourar.

O gráfico obtido no seu `benchmark.py` é uma representação textbook (de livro didático) do custo algorítmico $O(2^n)$. Ele valida não apenas a correção da sua implementação recursiva (que segue rigidamente a matemática esperada), mas também serve como um alerta visual sobre os perigos da explosão combinatória na ciência da computação. Algoritmos exponenciais são viáveis apenas para pequenas entradas de dados.

6. Conclusão

O presente projeto demonstrou com êxito a aplicação de conceitos de Matemática Discreta e de estruturas de dados por meio da implementação do clássico quebra-cabeça da Torre de Hanói. Através do desenvolvimento de uma aplicação interativa em Python, foi possível não apenas simular as regras físicas do jogo, mas também integrar um solucionador automático estritamente baseado em lógica recursiva. A modelagem orientada a

objetos, evidenciada por classes de controle como GameEngine e HanoiSolver , permitiu uma separação clara entre as regras de validação do tabuleiro e a renderização visual animada, resultando em um sistema robusto e responsivo.

Além da construção da interface, o estudo cumpriu o papel fundamental de analisar o comportamento assintótico do algoritmo. A bateria de testes e o levantamento de dados empíricos gerados pelo código de benchmark comprovaram matematicamente a complexidade de tempo exponencial do problema, denotada por $O(2^n)$. A convergência perfeita entre o modelo matemático preditivo e os resultados de execução evidenciou de forma prática os limites físicos de processamento e memória (call stack) ao lidar com o crescimento explosivo de operações. Isso reforça de maneira visual e concreta o alerta sobre a explosão combinatória, provando que algoritmos com essa complexidade são viáveis apenas para pequenas entradas de dados.

Em suma, o trabalho consolida o entendimento teórico e prático sobre recursividade, gerenciamento assíncrono de eventos e otimização de código. A transição de um modelo algorítmico abstrato para uma ferramenta visual e analisável cumpre o objetivo de demonstrar como o planejamento de movimentos atua em cenários de restrição de engenharia. Para uma visualização completa do código e das implementações, você pode acessar o repositório do projeto através do seguinte link: ***Clique aqui para acessar o repositório.***